

Bài giảng 2:

Phân loại ảnh bằng bộ phân loại tuyến tính

Hành chính: Nhiệm vụ 1

Hàng sẽ được gửi đi HÔM NAY, ngày 2/4, hạn chót ngày 16/4 là 23:59.

- Thuật toán K-Hàng xóm gần nhất
- Bộ phân loại tuyến tính: Softmax
- Mạng nơ-ron hai lớp
- Đặc điểm hình ảnh
- Mạng nơ-ron sâu và các thuật toán tối ưu hóa

Hành chính: Dự án khóa học

Hạn nộp đề xuất dự án: 23/4 (Thứ Năm) 23:59. Chúng tôi sẽ công bố chi tiết đề xuất vào ngày 16/4.

Hãy liên hệ với chúng tôi qua Ed. Mỗi nhóm dự án cuối cùng sẽ được chỉ định một trợ giảng để giải đáp các câu hỏi sau khi đề xuất dự án được nộp.

Hãy sử dụng [chủ đề Ed này](#) để tìm kiếm đối tác dự án. Chúng tôi cũng sẽ đăng tải một biểu mẫu Google Form để tìm kiếm đối tác dự án sau này.

"Liệu X có phải là một dự án hợp lệ cho môn 231n không?" --- Bài đăng riêng của Ed / Giờ làm việc của trợ giảng

Thông tin chi tiết hơn có trên trang web.

Quản trị: Các phần thảo luận

Thứ Sáu, từ 12:30 trưa đến 1:20 chiều, trực tiếp tại Hội trường NVIDIA, hoặc trực tuyến qua Zoom (bản ghi sẽ được cung cấp).

Python / Numpy, Google Colab

Chương trình học

Những kiến thức cơ bản về học sâu

Các phương pháp tiếp cận dựa trên dữ liệu

Phân loại tuyến tính

K-Hàng xóm gần nhất

Hàm mất mát

Tối ưu hóa

Lan truyền ngược

Mạng nơ-ron đa lớp

Mạng thần kinh

Chức năng kích hoạt

Tăng cường dữ liệu

Nhận thức và hiểu biết về thế giới thị giác

Chuyển giao học tập

Trình tối ưu hóa

Phép tích chập

PyTorch

Mạng nơ-ron hồi quy (RNN) / Cơ chế chú ý (Attention) / Máy biến áp (Transformers)

Lớp chuẩn hóa

Thiết kế kiến trúc

Hiểu video

Thị giác và Ngôn ngữ

Thị giác 3D

Phát hiện và phân đoạn đối tượng

Tái cấu trúc và tương tác với thế giới thị giác

Chuyển đổi phong cách

Mô hình tạo sinh

Học tập tự giám sát

Tạo ảnh

Robot và Trí tuệ nhân tạo thể hiện

Trí tuệ nhân tạo lấy con người làm trung tâm

Công bằng và đạo đức

Phân loại hình ảnh

Một nhiệm vụ cốt lõi trong thị giác máy tính

Hôm nay:

- Nhiệm vụ phân loại hình ảnh
- Hai phương pháp cơ bản dựa trên dữ liệu để phân loại hình ảnh
 - Thuật toán K-nearest neighbor và bộ phân loại tuyến tính

Phân loại ảnh: Một nhiệm vụ cốt lõi trong thị giác máy tính



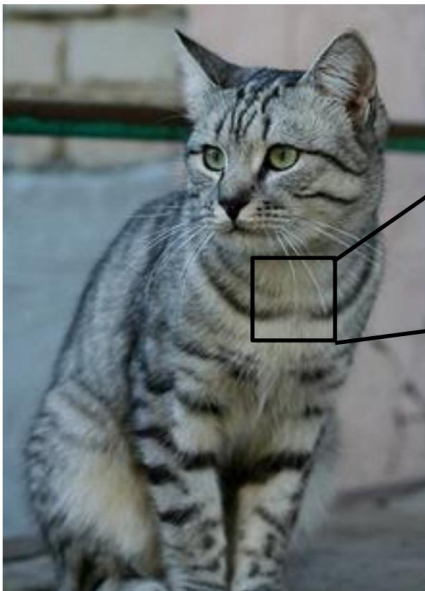
[Hình ảnh này bởi Niki Itz](#)
được cấp phép theo [CC-BY 2.0](#)

(giả sử đã cho một tập hợp các nhãn khả thi)
{chó, mèo, xe tải, máy bay, ...}



con mèo

Vấn đề: Khoảng cách ngữ nghĩa



[Hình ảnh này bởi Niki Itz](#)
được cấp phép theo [CC-BY 2.0](#)

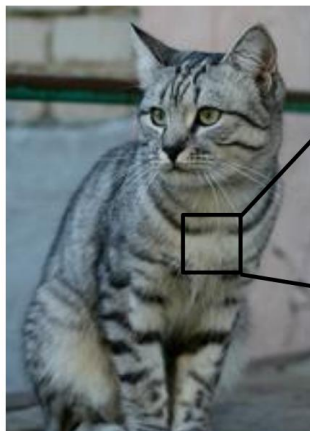
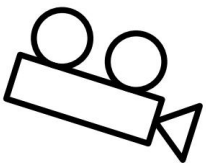
```
[[105 112 100 111 104 99 106 99 96 103 112 119 104 97 93 87]
 [ 91 98 102 106 104 79 98 103 99 105 123 136 110 105 94 85]
 [ 76 85 90 105 128 105 87 96 95 99 115 112 106 103 99 85]
 [ 99 81 81 93 120 131 127 100 95 98 102 99 96 93 101 94]
 [106 91 61 64 69 91 88 85 101 107 109 98 75 84 96 95]
 [114 100 85 55 55 69 64 54 64 87 112 129 98 74 84 91]
 [133 137 147 103 65 81 80 65 52 54 74 84 102 93 85 82]
 [128 137 144 140 109 95 86 70 62 65 63 63 60 73 86 101]
 [125 133 148 137 119 121 117 94 65 79 80 65 54 64 72 90]
 [127 125 131 147 133 127 126 131 111 96 89 75 61 64 72 84]
 [115 114 109 123 150 148 131 118 113 109 100 92 74 65 72 78]
 [ 89 93 90 97 100 147 131 118 113 114 113 109 106 95 77 80]
 [ 63 77 86 81 77 79 102 123 117 115 117 125 125 130 115 87]
 [ 62 65 82 89 78 71 80 101 124 126 119 101 107 114 131 119]
 [ 63 65 75 88 89 71 62 81 120 138 135 105 81 98 110 110]
 [ 87 65 71 87 106 95 69 45 76 130 126 107 92 94 105 112]
 [118 97 82 86 117 123 116 66 41 51 95 93 89 95 102 107]
 [164 146 112 80 82 120 124 104 76 40 45 66 88 101 102 109]
 [157 170 157 120 93 86 114 132 112 97 69 55 70 82 99 94]
 [130 128 134 161 139 100 109 118 121 134 114 87 65 53 69 86]
 [120 112 96 117 150 144 120 115 104 107 102 93 87 81 72 79]
 [123 107 96 86 83 112 153 149 122 109 104 75 80 107 112 99]
 [122 121 102 80 82 86 94 117 145 148 153 102 50 78 92 107]
 [122 164 148 103 71 56 78 83 93 103 119 139 102 61 69 84]]
```

Những gì máy tính nhìn thấy

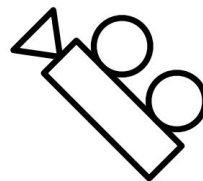
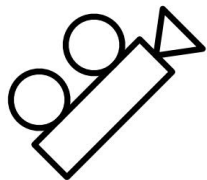
Một ảnh là một tensor gồm các số nguyên
nằm trong khoảng $[0, 255]$:

ví dụ $800 \times 600 \times 3$
(3 kênh RGB)

Thách thức: Sự khác biệt về quan điểm



11045	1132	1080	111	104	99	106	99	96	103	112	119	104	97	93	872
91	96	102	106	104	79	88	103	99	105	123	136	118	109	94	853
76	85	90	105	110	105	87	96	95	99	115	112	100	103	99	853
99	85	81	93	120	131	127	100	95	98	102	99	96	93	101	842
106	91	61	64	69	91	88	85	101	107	109	98	75	84	96	953
114	100	85	55	65	69	64	54	64	67	112	129	98	74	84	913
1333	137	147	103	65	81	88	65	52	54	74	84	102	93	85	823
120	137	144	140	109	95	86	78	82	85	63	63	88	73	86	1013
125	133	140	137	119	121	117	84	65	79	88	65	64	64	72	903
127	125	131	147	133	127	126	131	111	96	89	75	61	64	72	843
115	114	109	123	158	140	131	118	113	109	100	92	74	65	72	783
88	93	88	97	100	147	132	110	113	114	113	100	106	95	77	883
63	77	86	81	77	79	102	123	117	115	117	125	129	138	115	873
62	85	82	89	78	71	88	101	124	126	119	101	107	114	131	1103
63	85	79	88	89	71	62	83	120	130	135	101	61	88	110	1103
87	85	71	87	100	95	69	45	76	130	126	107	92	94	105	1123
110	97	82	86	117	123	136	66	41	53	95	93	89	95	102	1073
104	146	113	89	62	120	124	104	76	48	45	68	88	101	102	1003
117	170	117	120	63	86	114	132	112	87	69	55	70	82	99	843
130	120	134	101	139	100	109	118	121	134	114	87	65	53	69	863
120	112	96	117	158	144	120	115	104	107	102	93	87	81	72	793
121	107	96	83	112	153	149	127	100	104	75	68	87	112	99	1123
122	121	102	88	82	86	94	117	143	140	153	102	58	78	92	1073
122	104	140	101	71	56	79	83	93	103	119	130	102	61	89	8433



Tất cả các điểm ảnh đều thay
đổi khi máy ảnh di chuyển!

Thách thức: Chiều sáng



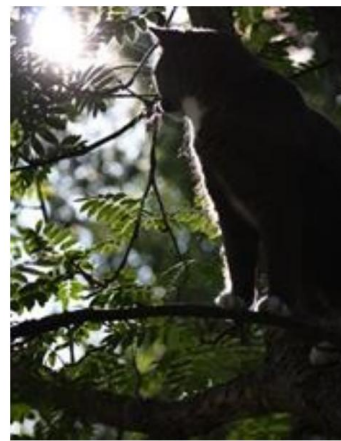
[Hình ảnh này là CC0 1.0](#) phạm vi công cộng



[Hình ảnh này là CC0 1.0](#) phạm vi công cộng



[Hình ảnh này là CC0 1.0](#) phạm vi công cộng



[Hình ảnh này là CC0 1.0](#) phạm vi công cộng

Thách thức: Hình nền lộn xộn



[Hình ảnh này](#) là [CC0 1.0](#) phạm vi công cộng



[Hình ảnh này](#) là [CC0 1.0](#) phạm vi công cộng

Thách thức: Tắc nghẽn



Hình ảnh này là [CC0 1.0](#) phạm vi công cộng



Hình ảnh này là [CC0 1.0](#) phạm vi công cộng

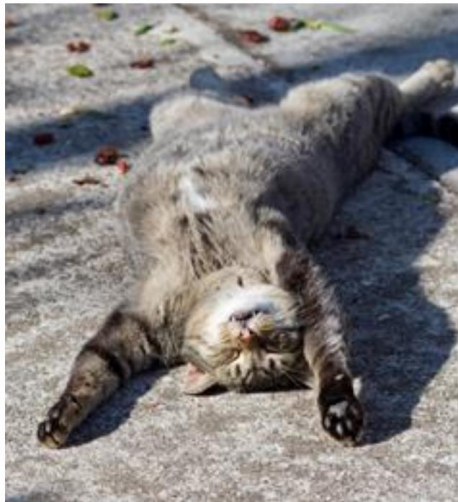


Hình ảnh này bởi [Johnson](#) được cấp phép theo [CC-BY 2.0](#)

Thách thức: Biến dạng



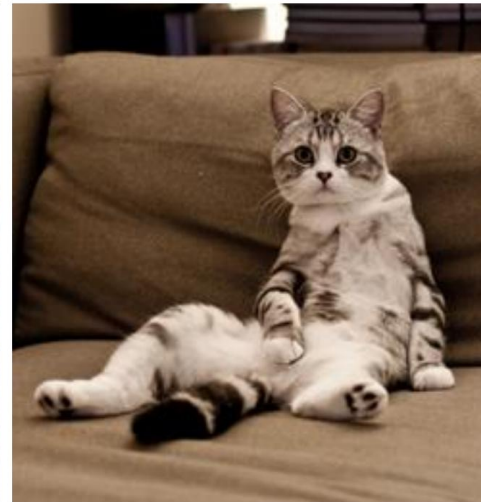
[Hình ảnh này bởi Umberto Salvagnin](#) được cấp
phép theo [CC-BY 2.0](#)



[Hình ảnh này bởi Umberto Salvagnin](#) được cấp
phép theo [CC-BY 2.0](#)



[Hình ảnh này bởi sa re bear r](#)
được cấp phép theo [CC-BY 2.0](#)



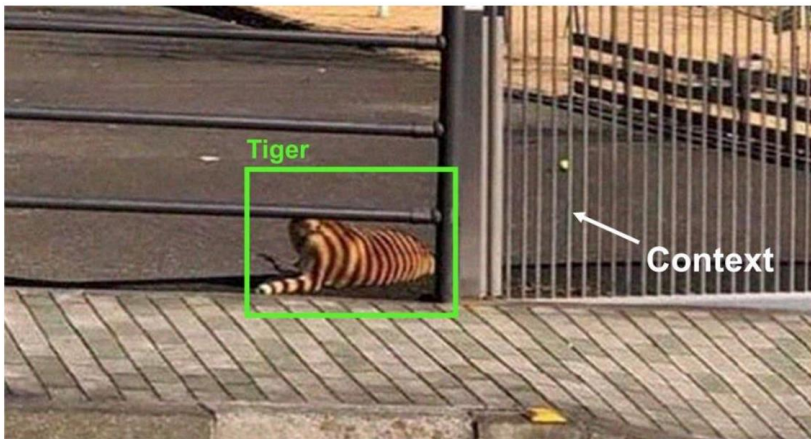
[Hình ảnh này bởi Tom Thai](#) được cấp phép
theo [CC-BY 2.0](#)

Thách thức: Sự khác biệt trong cùng một lớp



[Hình ảnh này](#) là [CC0 1.0](#) phạm vi công cộng

Thách thức: Bối cảnh



Nguồn ảnh: https://www.linkedin.com/posts/ralph-aboujaoude-diaz-40838313_technology-artificialintelligence-computervision-activity-6912446088364875776-h-Iq?utm_source=linkedin_share&utm_medium=member_desktop_web

Các thuật toán thị giác máy tính hiện đại



[Hình ảnh này](#) là [CC0 1.0](#) phạm vi công cộng

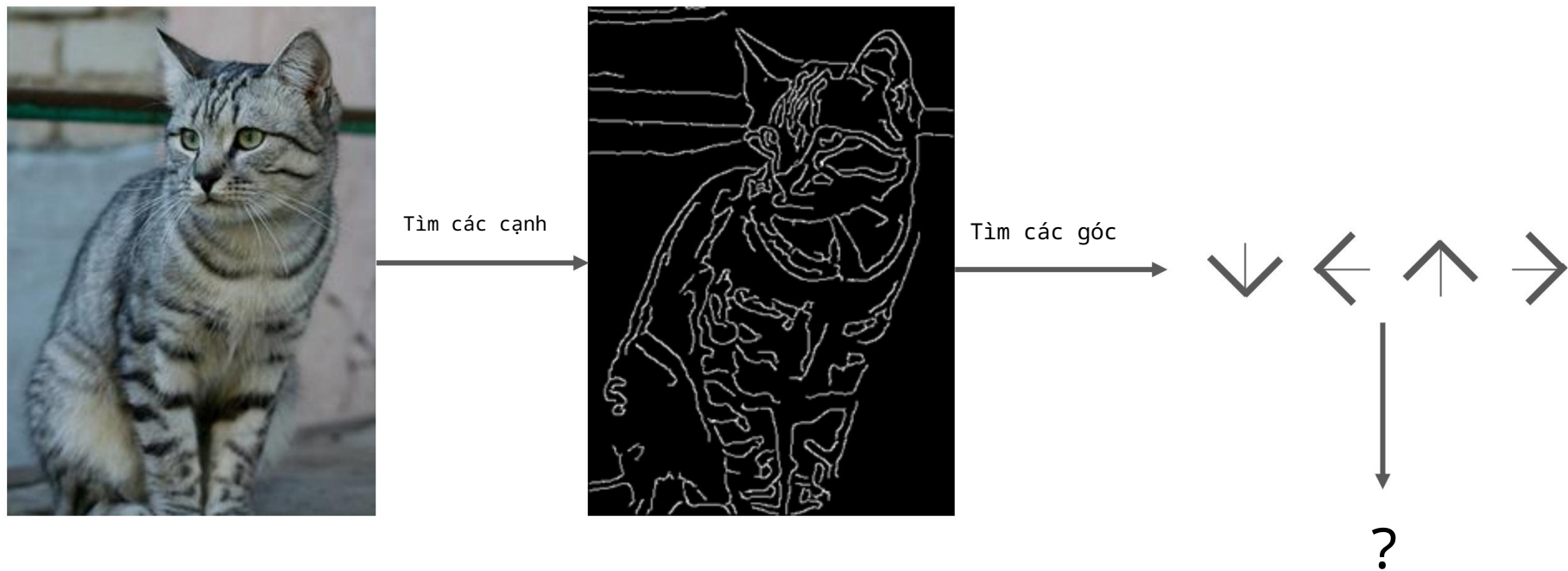
Bộ phân loại hình ảnh

```
def classify_image(image):  
    # Some magic here?  
    return class_label
```

Khác với việc ví dụ như sắp xếp một danh sách các số,

Không có cách nào rõ ràng để lập trình sẵn thuật toán nhận dạng mèo hoặc các lớp khác.

Đã có những nỗ lực được thực hiện.



John Canny, "Một phương pháp tính toán để phát hiện cạnh", IEEE TPAMI 1986

Học máy: Phương pháp dựa trên dữ liệu 1. Thu thập tập dữ liệu hình ảnh và nhãn 2. Sử dụng thuật toán học máy để huấn luyện bộ phân loại 3. Đánh giá bộ phân loại trên các hình ảnh mới

Ví dụ về tập dữ liệu huấn luyện

```
def train(images, labels):  
    # Machine learning!  
    return model
```

```
def predict(model, test_images):  
    # Use model to predict labels  
    return test_labels
```

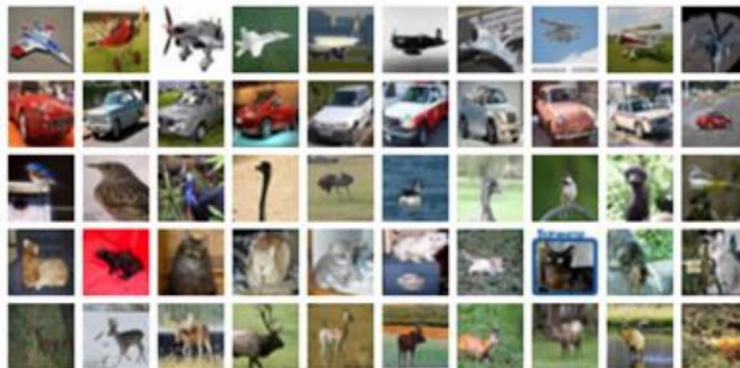
airplane

automobile

bird

cat

deer



Bộ phân loại lân cận gần nhất

Thuật toán phân loại đầu tiên: Nearest Neighbor

```
def train(images, labels):  
    # Machine learning!  
    return model
```



Ghi nhớ tất cả dữ liệu và nhãn.

```
def predict(model, test_images):  
    # Use model to predict labels  
    return test_labels
```



Dự đoán nhãn của hình ảnh huấn luyện tương tự nhất.

Thuật toán phân loại đầu tiên: Nearest Neighbor



Dữ liệu huấn luyện có nhãn



dữ liệu truy vấn

Số liệu khoảng cách

$$\left| \begin{array}{c} \text{tabby cat} \\ \text{cat} \end{array} \right|, \left| \begin{array}{c} \text{cat} \\ \text{cat} \end{array} \right| \rightarrow \mathbb{R}$$

Phương pháp đo khoảng cách để so sánh hình ảnh

Khoảng cách L1:
$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$

test image

56	32	10	18
90	23	128	133
24	26	178	200
2	0	255	220

-

training image

10	20	24	17
8	10	89	100
12	16	178	170
4	32	233	112

=

pixel-wise absolute value differences

46	12	14	1
82	13	39	33
12	10	0	30
2	32	22	108

thêm vào → 456

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

Bộ phân loại lân cận gần nhất

```
import numpy as np
```

```
class NearestNeighbor:
```

```
    def __init__(self):
```

```
        pass
```

```
    def train(self, X, y):
```

```
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
```

```
        # the nearest neighbor classifier simply remembers all the training data
```

```
        self.Xtr = X
```

```
        self.ytr = y
```

```
    def predict(self, X):
```

```
        """ X is N x D where each row is an example we wish to predict label for """
```

```
        num_test = X.shape[0]
```

```
        # lets make sure that the output type matches the input type
```

```
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)
```

```
        # loop over all test rows
```

```
        for i in xrange(num_test):
```

```
            # find the nearest training image to the i'th test image
```

```
            # using the L1 distance (sum of absolute value differences)
```

```
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
```

```
            min_index = np.argmin(distances) # get the index with smallest distance
```

```
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example
```

```
        return Ypred
```

Bộ phân loại lân cận gần nhất

Ghi nhớ dữ liệu huấn luyện

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

Bộ phân loại lân cận gần nhất

Đối với mỗi ảnh thử nghiệm:

Tìm hình ảnh tàu hỏa gần nhất

Dự đoán nhãn của ảnh gần nhất

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

Bộ phân loại lân cận gần nhất

Hỏi: Với N ví dụ, quá trình huấn luyện và dự đoán diễn ra nhanh như thế nào?

Đáp án: Tàu $O(1)$,
dự đoán $O(N)$

Điều này không tốt: chúng ta cần các bộ phân loại có tốc độ dự đoán nhanh; tốc độ huấn luyện chậm thì chấp nhận được.

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

Bộ phân loại lân cận gần nhất

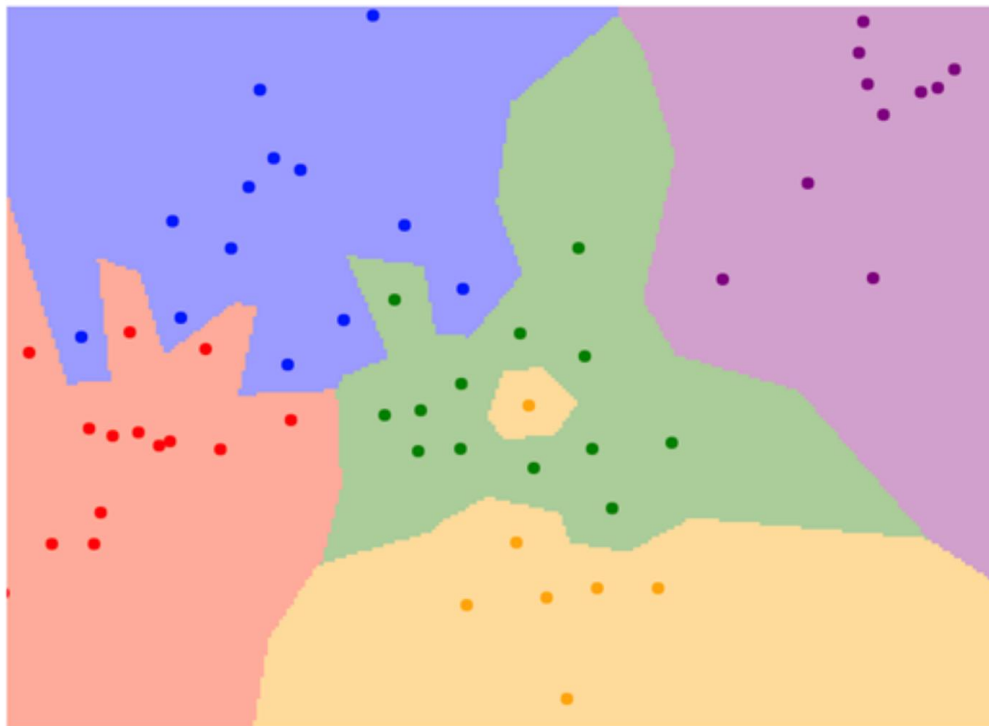
Có rất nhiều phương pháp tìm lân cận gần đúng nhanh/xấp xỉ (ngoài phạm vi của 231N!).

Một cách triển khai tốt:

<https://github.com/facebookresearch/faiss>

Johnson và cộng sự, "Tìm kiếm sự tương đồng quy mô tỷ với GPU", arXiv 2017

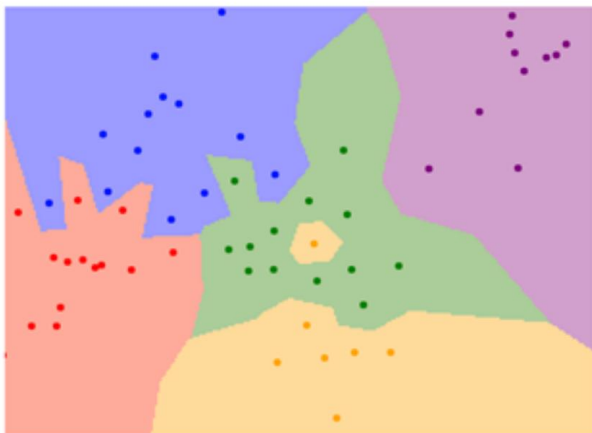
Nó trông như thế nào?



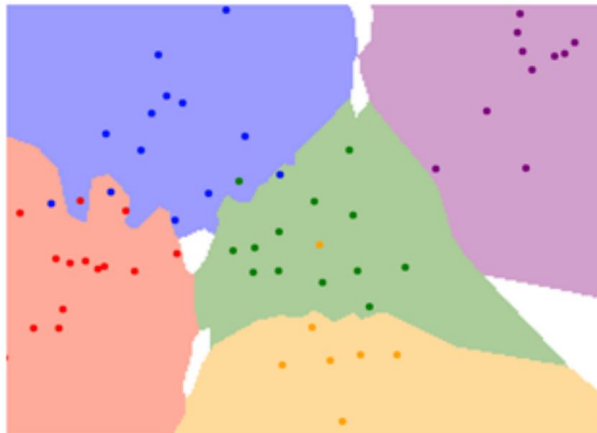
1-hàng xóm gần nhất

K-Hàng xóm gần nhất

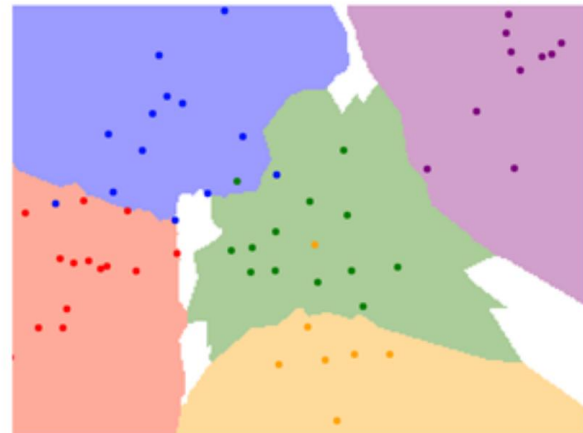
Thay vì sao chép nhãn từ điểm lân cận gần nhất,
hãy lấy phiếu bầu đa số từ K điểm gần nhất.



$K = 1$



$K = 3$

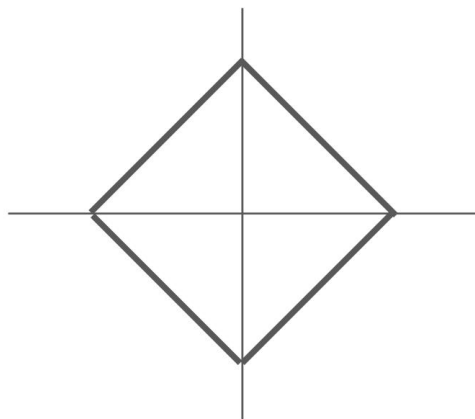


$K = 5$

K-Láng giếng gần nhất: Phương pháp đo khoảng cách

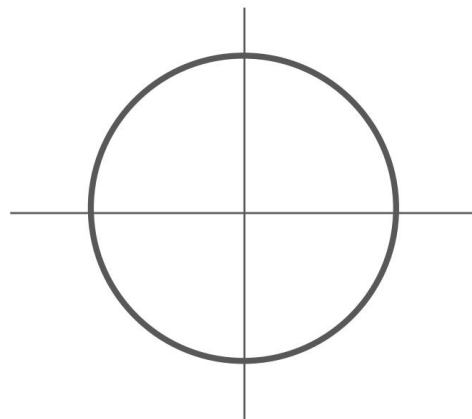
Khoảng cách L1 (Manhattan)

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



Khoảng cách L2 (Euclidean)

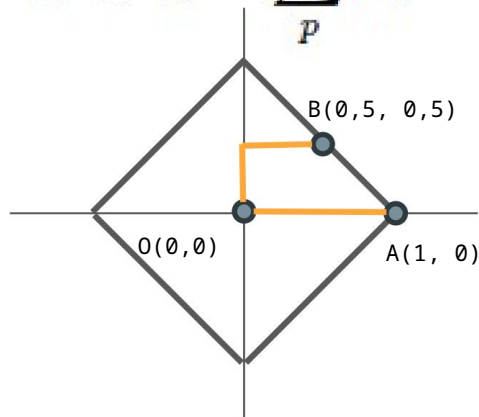
$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$



K-Hàng xóm gần nhất: Phương pháp đo khoảng cách - Ví dụ

L1 Khoảng cách: Đo khoảng cách bằng cách đi chuyển dọc theo các đường lưới (giống như đi bộ trong một thành phố có các khối nhà hình vuông).

$$d_1(I_1, I_2) = \sum_P |I_1^P - I_2^P|$$



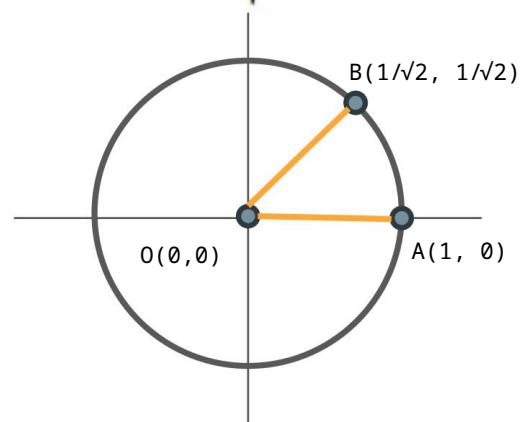
$$d_1(O, A) = |0-1| + |0-0| = 1$$

$$d_1(O, B) = |0-0.5| + |0-0.5| = 0.5 + 0.5 = 1$$

$$d_1(O, A) = d_1(O, B) = 1$$

Khoảng cách L2: Đo khoảng cách đường thẳng (theo đường chim bay).

$$d_2(I_1, I_2) = \sqrt{\sum_P (I_1^P - I_2^P)^2}$$



$$d_2(O, A) = \sqrt{(0-1)^2 + (0-0)^2} = \sqrt{1} = 1$$

$$d_2(O, B) = \sqrt{(0-1/\sqrt{2})^2 + (0-1/\sqrt{2})^2} = \sqrt{1/2 + 1/2} = \sqrt{1} = 1$$

$$d_2(O, A) = d_2(O, B) = 1$$

K-Láng giềng gần nhất: Phương pháp đo khoảng cách

Khoảng cách L1 (Manhattan)

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



K = 1

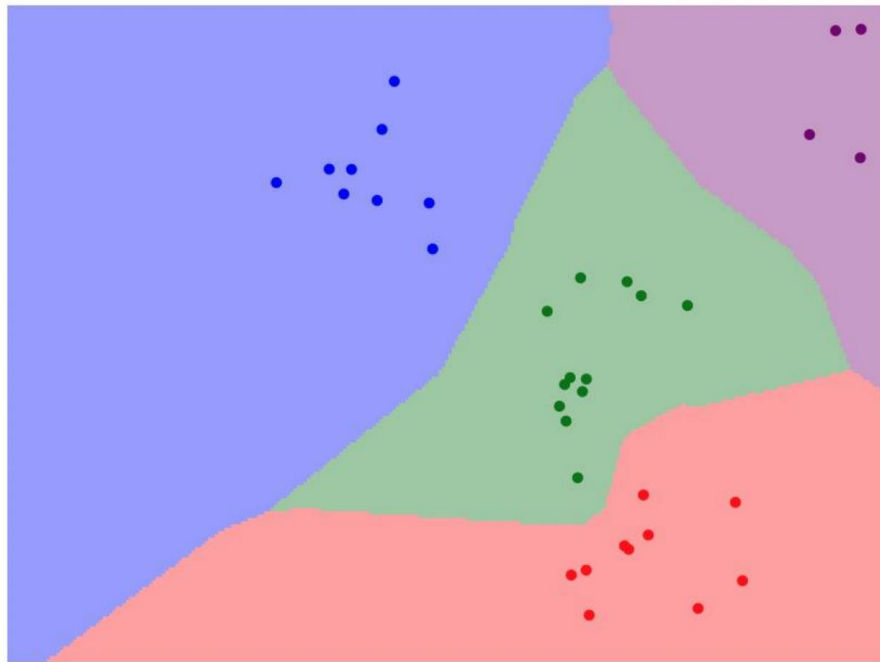
Khoảng cách L2 (Euclidean)

$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$



K = 1

K-Nearest Neighbors: Hãy tự mình thử!



<http://vision.stanford.edu/teaching/cs231n-demos/knn/>

Siêu tham số

Giá trị k tốt nhất nên sử dụng là bao nhiêu?

Khoảng cách tốt nhất nên sử dụng là bao nhiêu?

Đây là các siêu tham số: những lựa chọn về chính các thuật toán.

Rất phụ thuộc vào vấn đề/tập dữ liệu.

Phải thử hết tất cả và xem cái nào hiệu quả nhất.

Thiết lập siêu tham số

Ý tưởng #1: Chọn các siêu tham số hoạt động tốt nhất trên dữ liệu huấn luyện.

xe lửa

Thiết lập siêu tham số

Ý tưởng #1: Chọn các siêu tham số hoạt động tốt nhất trên dữ liệu huấn luyện.

SAI: $K = 1$ luôn hoạt động hoàn hảo trên dữ liệu huấn luyện

xe lửa

Thiết lập siêu tham số

Ý tưởng #1: Chọn các siêu tham số hoạt động tốt nhất trên dữ liệu huấn luyện.

SAI: $K = 1$ luôn hoạt động hoàn hảo trên dữ liệu huấn luyện

xe lửa

Ý tưởng #2: Chọn các siêu tham số hoạt động tốt nhất trên dữ liệu thử nghiệm.

xe lửa

Bài kiểm tra

Thiết lập siêu tham số

Ý tưởng #1: Chọn các siêu tham số hoạt động tốt nhất trên dữ liệu huấn luyện.

SAI: $K = 1$ luôn hoạt động hoàn hảo trên dữ liệu huấn luyện

xe lửa

Ý tưởng #2: Chọn các siêu tham số hoạt động tốt nhất trên dữ liệu thử nghiệm.

ĐIỂM TRỪ: Không biết thuật toán sẽ hoạt động như thế nào trên dữ liệu mới.

xe lửa

Bài kiểm tra

Đừng bao giờ làm điều này!

Thiết lập siêu tham số

Ý tưởng #1: Chọn các siêu tham số hoạt động tốt nhất trên dữ liệu huấn luyện.

SAI: $K = 1$ luôn hoạt động hoàn hảo trên dữ liệu huấn luyện

xe lửa

Ý tưởng #2: Chọn các siêu tham số hoạt động tốt nhất trên dữ liệu thử nghiệm.

ĐIỂM TRỪ: Không biết thuật toán sẽ hoạt động như thế nào trên dữ liệu mới.

xe lửa

Bài kiểm tra

Ý tưởng #3: Chia dữ liệu thành tập huấn luyện và tập xác thực; chọn siêu tham số trên tập xác thực và đánh giá trên tập kiểm tra.

Tốt hơn!

xe lửa

xác thực

Bài kiểm tra

Thiết lập siêu tham số

xe lửa

Ý tưởng #4: Kiểm định chéo: Chia dữ liệu thành các phần nhỏ, thử nghiệm từng phần nhỏ như một phương pháp kiểm định và tính trung bình kết quả.

gấp 1	gấp 2	gấp 3	gấp 4	gấp 5
gấp 1	gấp 2	gấp 3	gấp 4	gấp 5
gấp 1	gấp 2	gấp 3	gấp 4	gấp 5
gấp 1	gấp 2	gấp 3	gấp 4	gấp 5
gấp 1	gấp 2	gấp 3	gấp 4	gấp 5

Bài kiểm tra

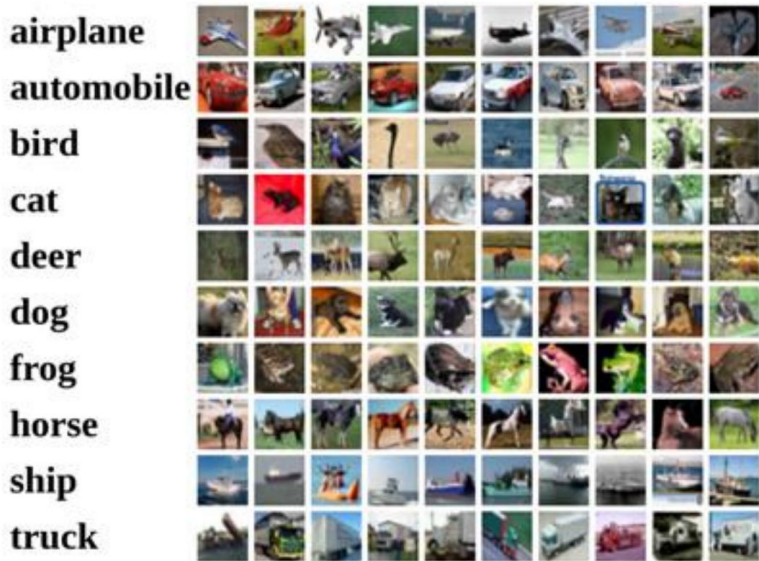
Hữu ích cho các tập dữ liệu nhỏ, nhưng không được sử dụng thường xuyên trong học sâu.

Ví dụ về bộ dữ liệu: CIFAR10

10 lớp học,

50.000 hình ảnh huấn luyện,

10.000 hình ảnh kiểm tra.



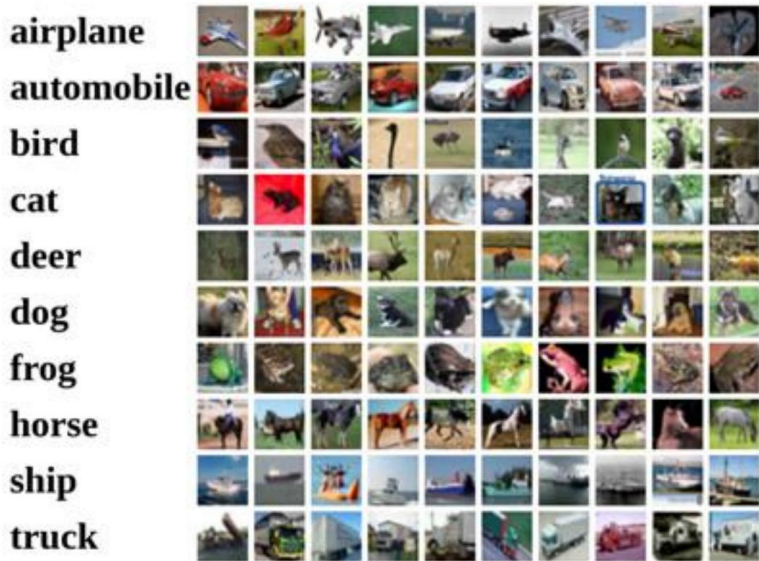
Alex Krizhevsky, "Học cách nhận biết nhiều lớp đặc điểm từ những hình ảnh nhỏ", Báo cáo kỹ thuật, 2009.

Ví dụ về bộ dữ liệu: CIFAR10

10 lớp học

50.000 hình ảnh huấn luyện

10.000 hình ảnh thử nghiệm

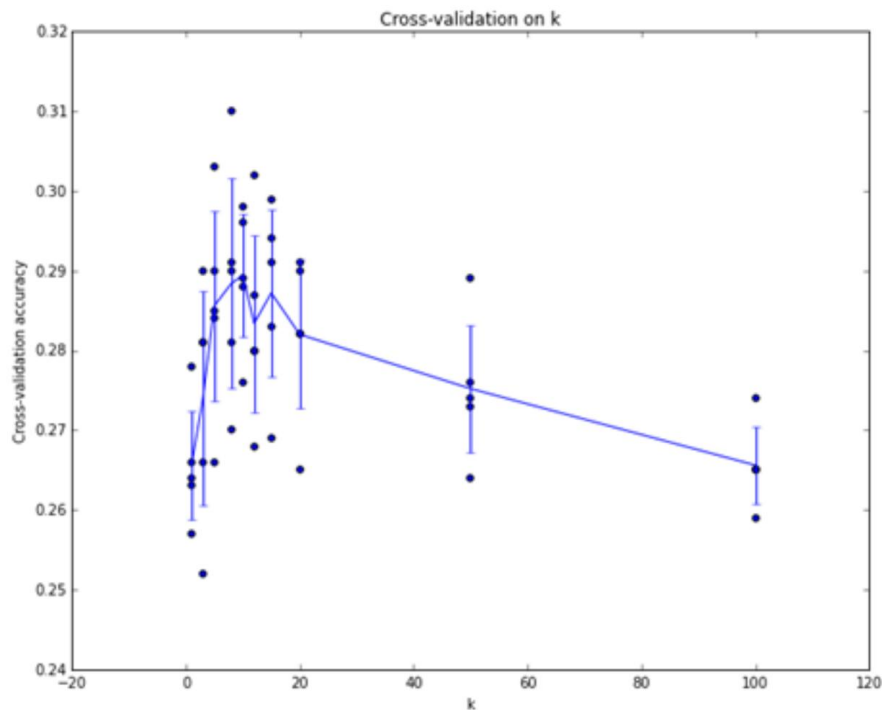


Ảnh thử nghiệm và các vùng lân cận gần nhất



Alex Krizhevsky, "Học cách nhận biết nhiều lớp đặc điểm từ những hình ảnh nhỏ", Báo cáo kỹ thuật, 2009.

Thiết lập siêu tham số



Ví dụ về
phương pháp kiểm định chéo
5 lần cho giá trị của k.

Mỗi điểm: đơn lẻ
kết quả.

Đường thẳng đi
qua giá trị trung bình, các
thanh ngang biểu thị độ
lệch chuẩn.

(Có vẻ như $k \approx 7$ là giá trị phù hợp
nhất với dữ liệu này)

Nó trông như thế nào?



Nó trông như thế nào?



Thuật toán k-Nearest Neighbor với khoảng cách pixel không bao giờ được sử dụng.

- Các chỉ số khoảng cách trên pixel không cung cấp thông tin hữu ích.

Hình ảnh gốc là CC0 Nguyên bản
phạm vi công cộng



Bị che khuất



Đã dịch chuyển (1 pixel)



Nhuộm màu



(Cả ba hình ảnh bên phải đều có khoảng cách pixel giống với hình ảnh bên trái)

K-Láng giềng gần nhất: Tóm tắt

Trong phân loại ảnh, chúng ta bắt đầu với một tập dữ liệu huấn luyện gồm các ảnh và nhãn, và phải dự đoán các nhãn trên tập dữ liệu kiểm tra.

Thuật toán phân loại K-Nearest Neighbors dự đoán nhãn dựa trên K ví dụ huấn luyện gần nhất.

Khoảng cách đo và K là các siêu tham số.

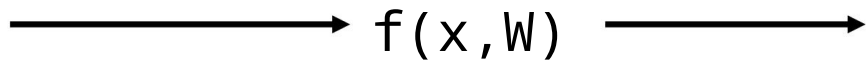
Chọn siêu tham số bằng cách sử dụng tập dữ liệu kiểm định.

Chỉ chạy trên tập dữ liệu thử nghiệm một lần duy nhất vào cuối cùng!

Bộ phân loại tuyến tính

Phương pháp tham số

Hình ảnh



10 con số thể hiện
điểm số của lớp



Tâm

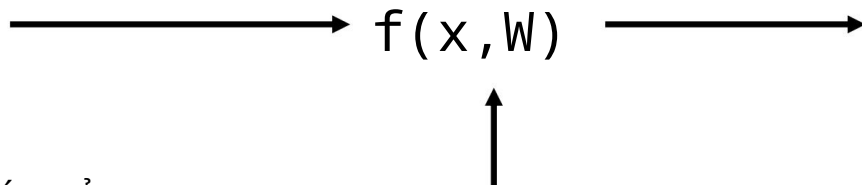
tham số hoặc
trọng số

Mảng gồm 32x32x3 số (tổng
cộng 3072 số)

Phương pháp tham số: Bộ phân loại tuyến tính

$$f(x, W) = W x$$

Hình ảnh



10 con số thể hiện
điểm số của lớp

Mảng gồm 32x32x3 số (tổng
cộng 3072 số)

Trọng số

tham số hoặc
trọng số

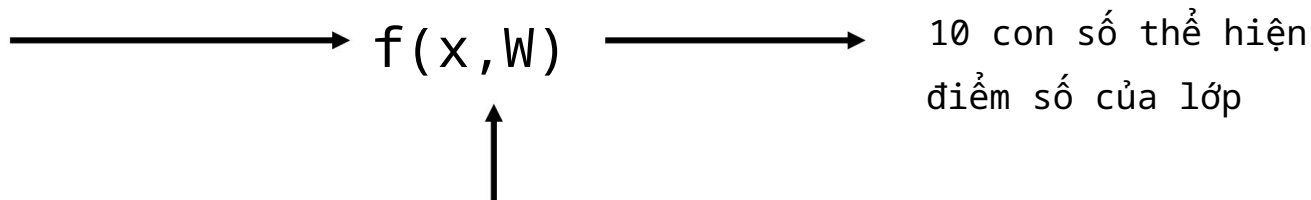
Phương pháp tham số: Bộ phân loại tuyến tính

Hình ảnh



$$f(x, W) = Wx$$

10x1 10x3072



Mảng gồm 32x32x3 số (tổng cộng 3072 số)

Tây

tham số hoặc
trọng số

Phương pháp tham số: Bộ phân loại tuyến tính

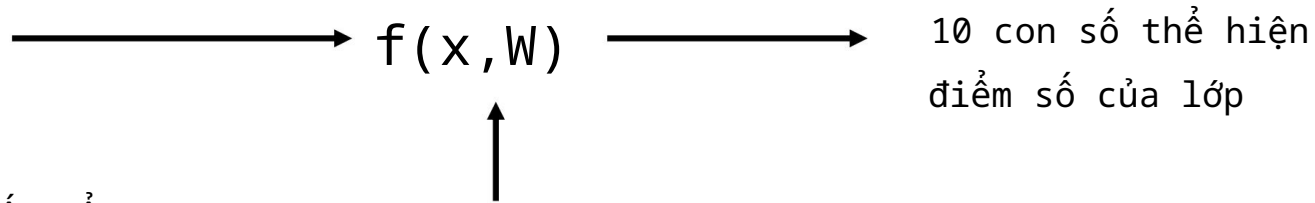
Hình ảnh



Mảng gồm $32 \times 32 \times 3$ số (tổng cộng 3072 số)

$$\boxed{f(x, W)} = \boxed{W} \boxed{x} + \boxed{b}$$

10×1 10×3072 3072×1 10×1



tham số hoặc
trọng số

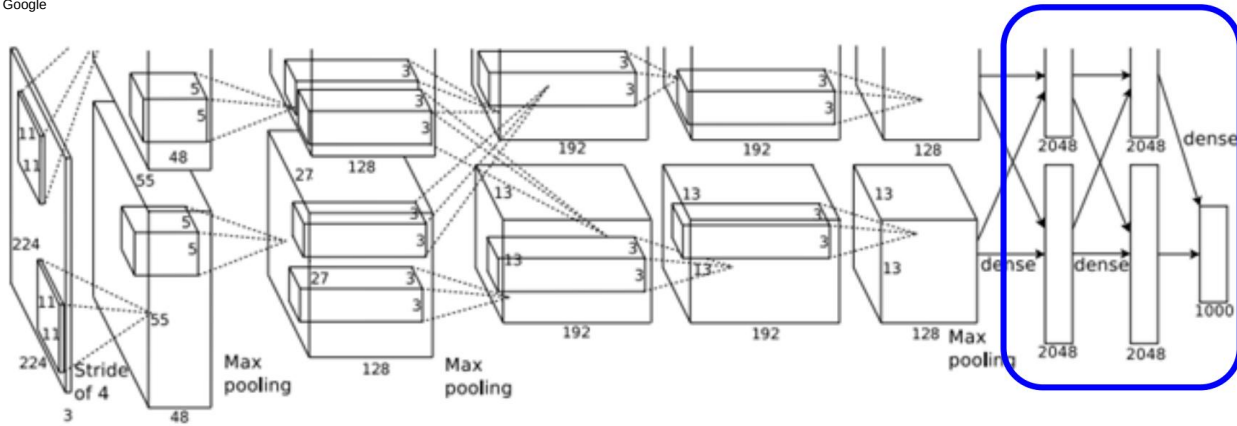
Mạng thần kinh

Bộ phân

loại tuyến tính

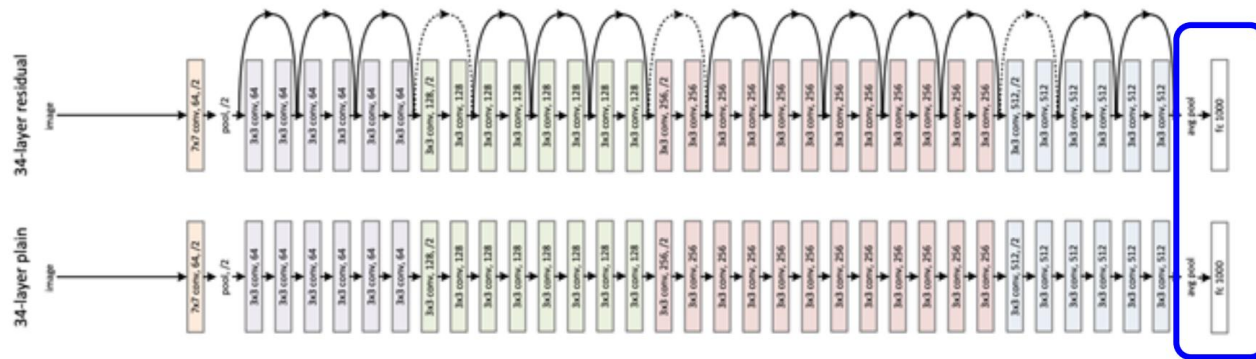


[Hình ảnh này là CC0 1.0](#) phạm vi công cộng



[Krizhevsky và cộng sự, 2012]

Lớp tuyến tính



[He và cộng sự, 2015]

Nhớ lại CIFAR10

airplane

automobile

bird

cat

deer

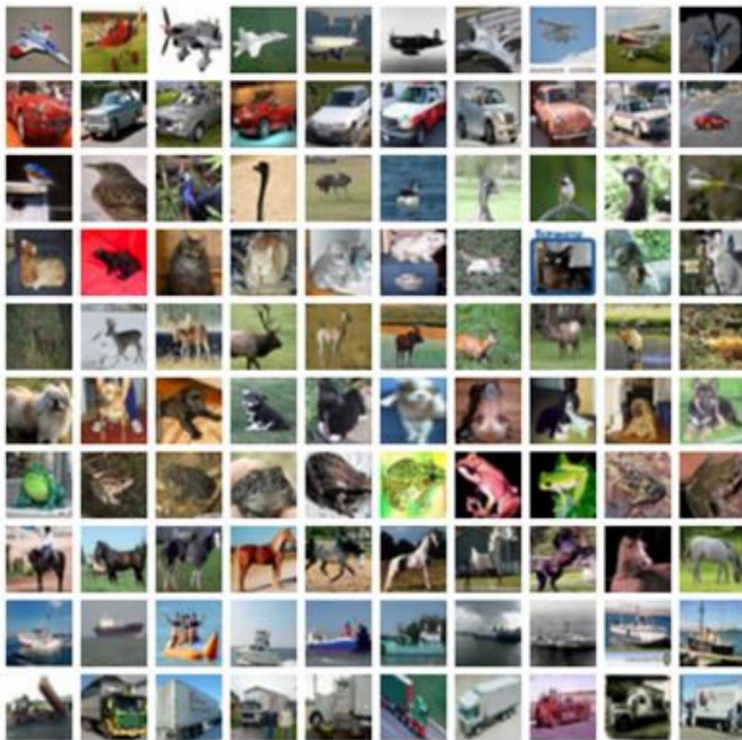
dog

frog

horse

ship

truck

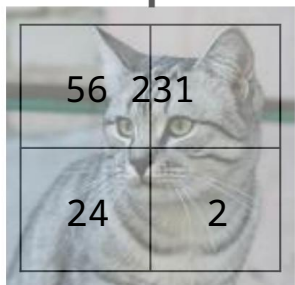


50.000 hình ảnh huấn luyện, mỗi
hình ảnh có kích thước 32x32x3.

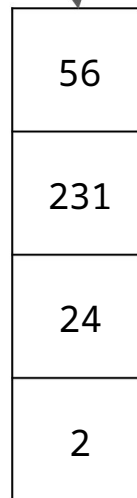
10.000 hình ảnh thử nghiệm.

Ví dụ với một hình ảnh có 4 pixel và 3 lớp (mèo/chó/tàu)

Chuyển đổi tensor thành vector



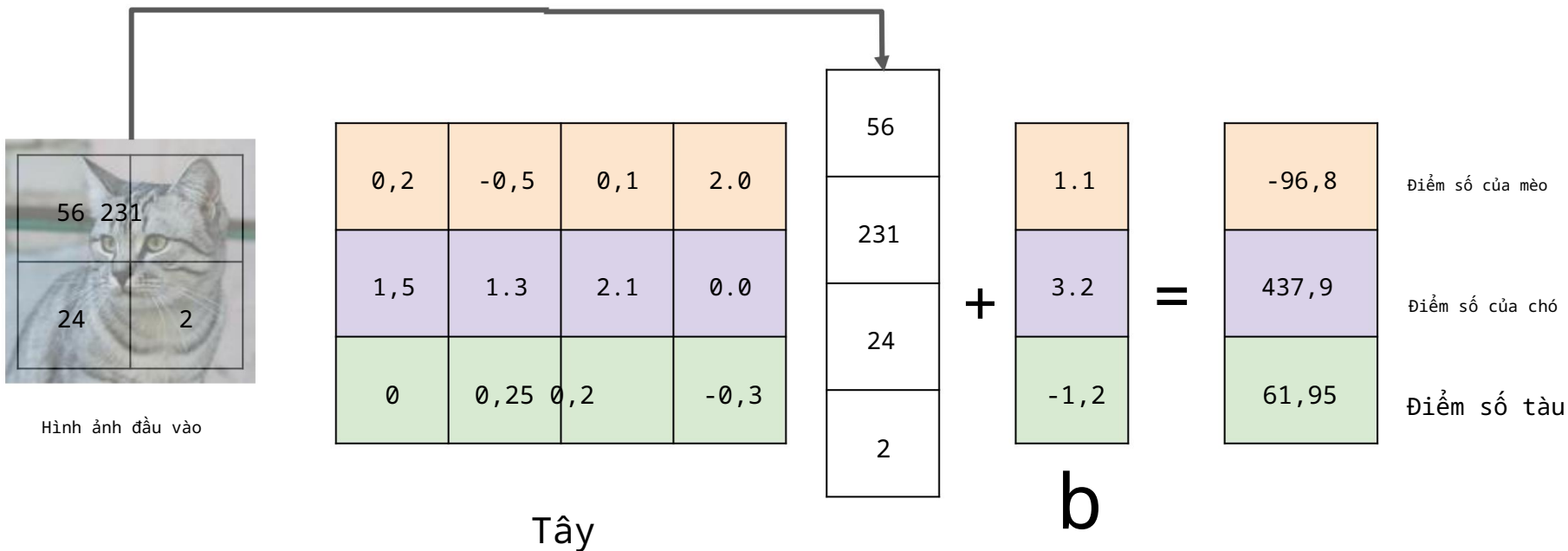
Hình ảnh đầu vào



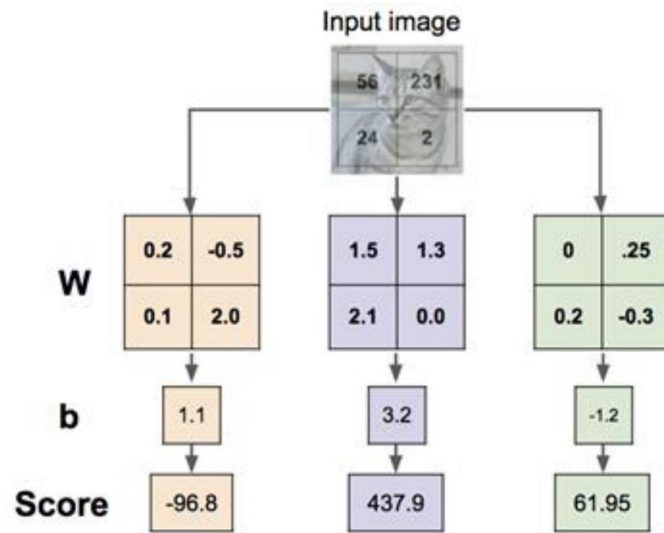
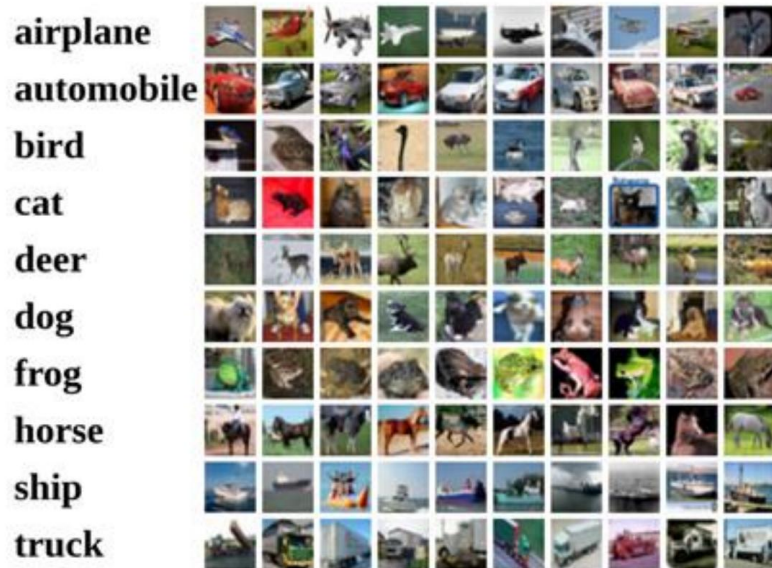
Ví dụ với một hình ảnh có 4 pixel và 3 lớp (mèo/chó/tàu)

Quan điểm đại số

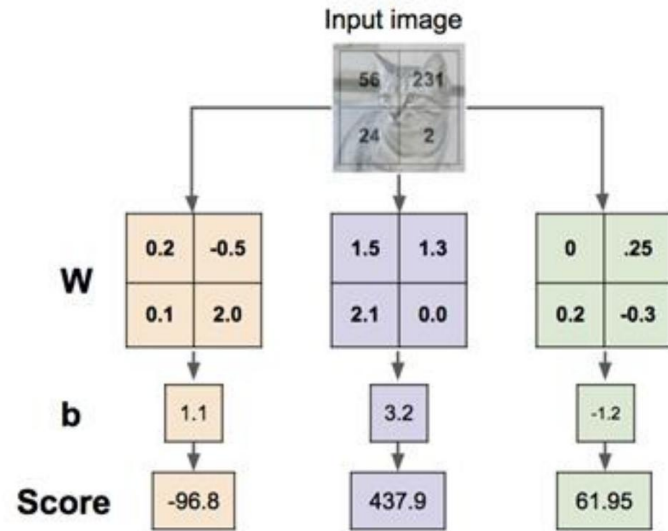
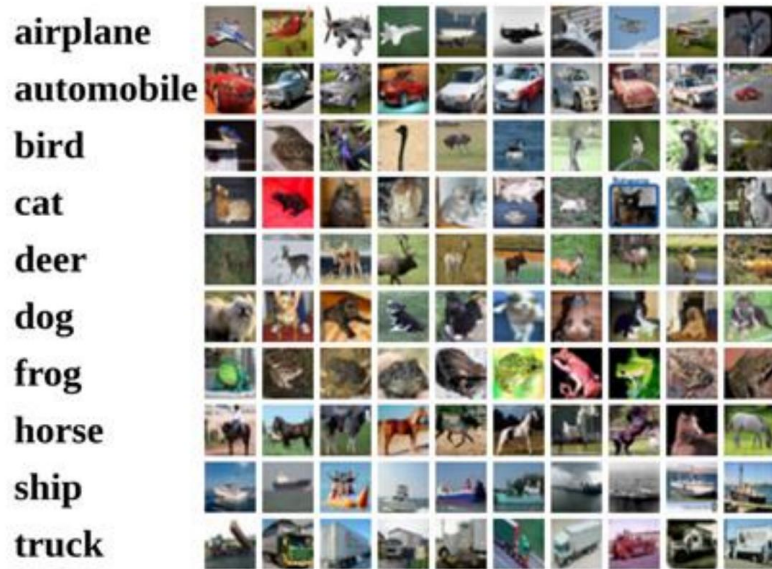
Chuyển đổi tensor thành vector



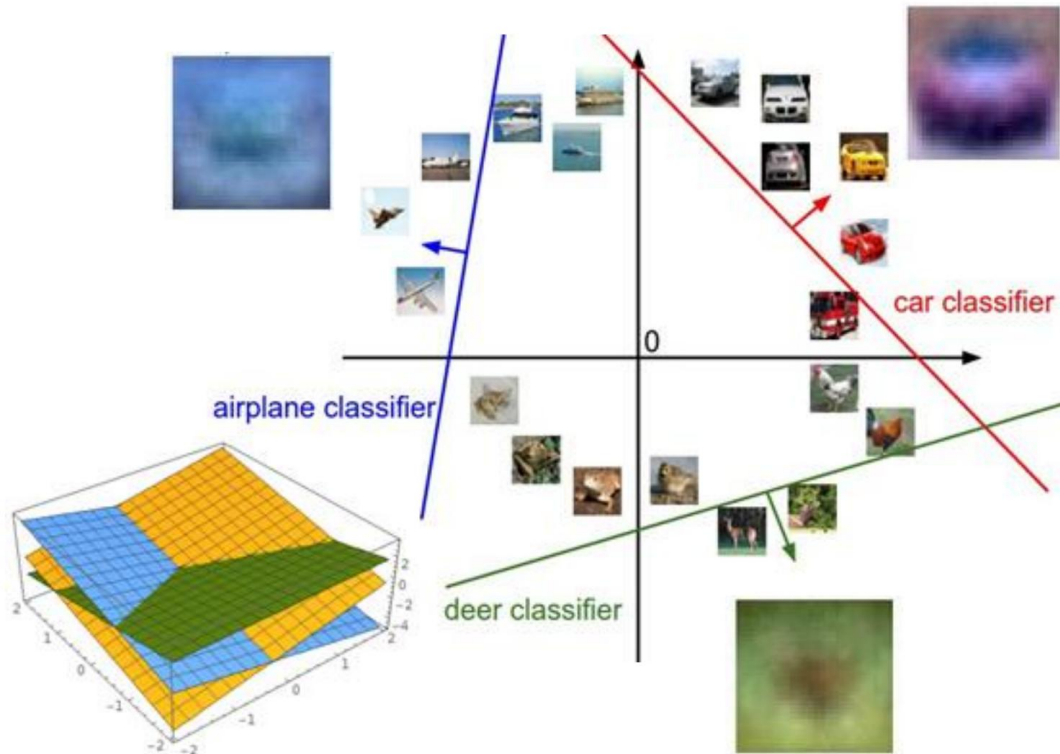
Giải thích bộ phân loại tuyến tính



Giải thích bộ phân loại tuyến tính: Góc nhìn trực quan



Giải thích bộ phân loại tuyến tính: Quan điểm hình học



Biểu đồ được tạo bằng [Wolfram Cloud](#)

$$f(x, W) = Wx + b$$



Mảng gồm 32x32x3 số
(Tổng cộng 3072 số)

[Hình ảnh con mèo bởi Nikita](#) được cấp phép theo [CC-BY 2.0](#)

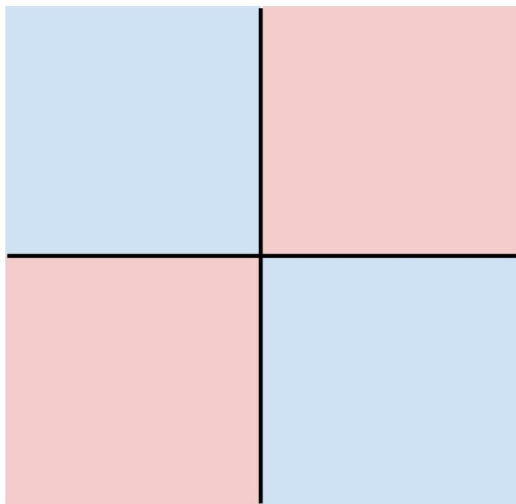
Các trường hợp khó đối với bộ phân loại tuyến tính

Lớp 1:

Góc phần tư thứ nhất và thứ ba

Lớp 2:

Góc phần tư thứ hai và thứ tư

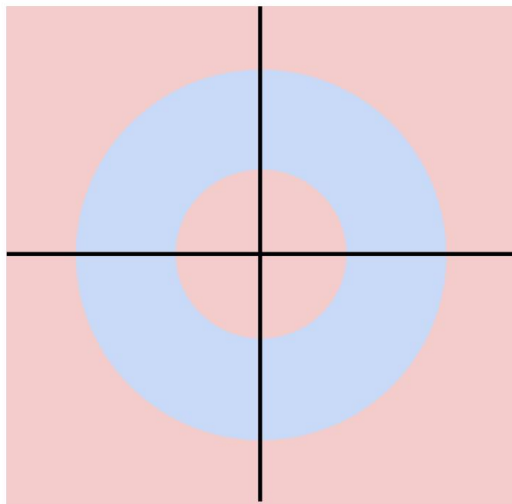


Lớp 1:

$1 \leq \text{Chuẩn L2} \leq 2$

Lớp 2:

Mọi thứ khác

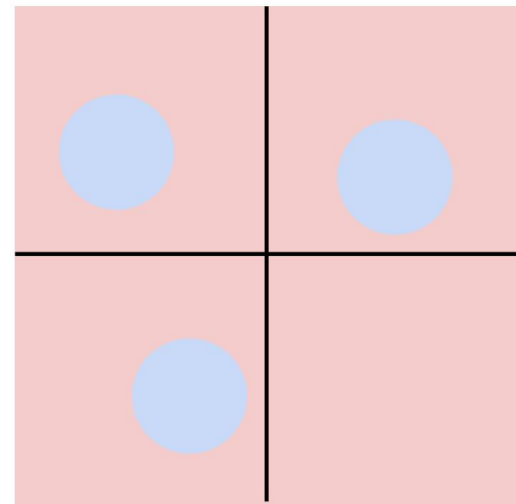


Lớp 1:

Ba chế độ

Lớp 2:

Mọi thứ khác



Bộ phân loại tuyến tính - Chọn một W tốt



airplane	-3.45	-0.51	3.42
automobile	-8.87	6.04	4.64
bird	0.09	5.31	2.65
cat	2.9	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	-4.34
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

1. Hãy định nghĩa một hàm mất mát định lượng mức độ không hài lòng của chúng ta với điểm số trên toàn bộ dữ liệu huấn luyện.

2. Hãy nghĩ ra một cách hiệu quả

Tìm các tham số sao cho hàm mất mát được tối thiểu hóa. (tối ưu hóa)

Bộ phân loại tuyến tính - Chọn một W tốt

$$f(x, W) = Wx$$

Hàm mất mát cho biết bộ phân loại hiện tại của chúng ta tốt đến mức nào.



airplane	-3.45	-0.51	3.42
automobile	-8.87	6.04	4.64
bird	0.09	5.31	2.65
cat	2.9	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	-4.34
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

[Hình ảnh con mèo](#) bởi [hikix](#) đã được cấp phép theo [CC-BY 2.0](#). [Hình ảnh xe hơi là cco](#) [1.0](#) [Hình ảnh](#) thuộc phạm vi công cộng. [Hình ảnh con ếch](#) thuộc phạm vi công cộng.

Bộ phân loại tuyến tính - Chọn một W tốt

$$f(x, W) = Wx$$

Hàm mất mát cho biết bộ phân loại hiện tại của chúng ta tốt đến mức nào.

Cho trước một tập dữ liệu các ví dụ

$$\{(x_i, y_i)\}_{i=1}^N$$

Trong đó, x_i hình ảnh là và y_i nhãn (số nguyên) là



airplane	-3.45	-0.51	3.42
automobile	-8.87	6.04	4.64
bird	0.09	5.31	2.65
cat	2.9	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	-4.34
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

[Hình ảnh con mèo](#) bởi [hik](#) đã được cấp phép theo [CC-BY 2.0](#). [Hình ảnh xe hơi](#) là [CCO](#). [Hình ảnh](#) thuộc phạm vi công cộng. [Hình ảnh con ếch](#) thuộc phạm vi công cộng.

Bộ phân loại tuyến tính - Chọn một W tốt



airplane	-3.45	-0.51	3.42
automobile	-8.87	6.04	4.64
bird	0.09	5.31	2.65
cat	2.9	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	-4.34
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

$$f(x, W) = Wx$$

Hàm mất mát cho biết bộ phân loại hiện tại của chúng ta tốt đến mức nào.

Cho trước một tập dữ liệu các ví dụ

$$\{(x_i, y_i)\}_{i=1}^N$$

Trong đó, x_i là gì và nhãn (số

y_i nguyên) là gì

Mức tổn thất trên toàn bộ tập dữ liệu là mức trung bình của tổn thất trên các ví dụ:

$$L = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i)$$

Bộ phân loại Softmax

Bộ phân loại Softmax (Hồi quy logistic đa thức)



Muốn diễn giải điểm số phân loại thô dưới dạng xác suất

con mèo	3.2
xe hơi	5.1
con ếch	-1,7

Bộ phân loại Softmax (Hồi quy logistic đa thức)



Muốn diễn giải điểm số phân loại thô dưới dạng xác suất

$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Chức năng

con mèo

3.2

xe hơi

5.1

con ếch

-1,7

Bộ phân loại Softmax (Hồi quy logistic đa thức)



Muốn diễn giải điểm số phân loại thô dưới dạng xác suất

$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Chức năng

Xác suất phải

lớn hơn hoặc bằng 0

con mèo

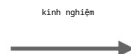
3.2

xe hơi

5.1

con ếch

-1,7



24,5

164.0

0,18

xác suất chưa

được chuẩn hóa

Bộ phân loại Softmax (Hồi quy logistic đa thức)



Muốn diễn giải điểm số phân loại thô dưới dạng xác suất

$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Chức năng

Xác suất phải
lớn hơn hoặc bằng 0

Xác suất
phải có tổng bằng 1

con mèo

3.2

xe hơi

5.1

con ếch

-1,7

kinh nghiệm

24,5

164.0

0,18

chuẩn hóa

0.13

0.87

0,00

xác suất chưa
được chuẩn hóa

xác suất

Bộ phân loại Softmax (Hồi quy logistic đa thức)



Muốn diễn giải điểm số phân loại thô dưới dạng xác suất

$$s = f(x_i; W)$$

Xác suất phải
lớn hơn hoặc bằng 0

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Chức năng

Xác suất
phải có tổng bằng 1

con mèo

xe hơi

con ếch

3.2

5.1

-1,7

kinh nghiệm

24,5

164.0

0,18

chuẩn hóa

0.13

0.87

0,00

Xác suất logarit
chưa chuẩn hóa / logit

xác suất chưa
được chuẩn hóa

xác suất

Bộ phân loại Softmax (Hồi quy logistic đa thức)



Muốn diễn giải điểm số phân loại thô dưới dạng xác suất

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Chức năng

Xác suất phải
lớn hơn hoặc bằng 0

Xác suất
phải có tổng bằng 1

$$L_i = -\log P(Y = y_i|X = x_i)$$

con mèo

3.2

xe hơi

5.1

con ếch

-1,7

kinh nghiệm

24,5

164.0

0,18

chuẩn hóa

0.13

0.87

0,00

$$\begin{aligned} \rightarrow L_i &= -\log(0.13) \\ &= 2,04 \end{aligned}$$

Xác suất logarit
chưa chuẩn hóa / logit

xác suất chưa
được chuẩn hóa

xác suất

Bộ phân loại Softmax (Hồi quy logistic đa thức)



Muốn diễn giải điểm số phân loại thô dưới dạng xác suất

$$s = f(x_i; W)$$

Xác suất phải
lớn hơn hoặc bằng 0

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Chức năng

Xác suất
phải có tổng bằng 1

$$L_i = -\log P(Y = y_i|X = x_i)$$

con mèo

3.2

xe hơi

5.1

con ếch

-1,7

kinh nghiệm

24,5

164.0

0,18

chuẩn hóa

0.13

0.87

0,00

$$\begin{aligned} \rightarrow L_i &= -\log(0.13) \\ &= 2,04 \end{aligned}$$

Ước lượng hợp lý tối đa

Chọn trọng số để tối đa hóa khả năng xảy ra của dữ liệu quan sát được.

(Xem CS 229 để biết thêm chi tiết)

Xác suất logarit
chưa chuẩn hóa / logit

xác suất chưa
được chuẩn hóa

xác suất

Bộ phân loại Softmax (Hồi quy logistic đa thức)



Muốn diễn giải điểm số phân loại thô dưới dạng xác suất

$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Chức năng

Xác suất phải
lớn hơn hoặc bằng 0

Xác suất
phải có tổng bằng 1

$$L_i = -\log P(Y = y_i | X = x_i)$$

con mèo

xe hơi

con ếch

3.2

5.1

-1,7

kinh nghiệm

24,5

164.0

0,18

chuẩn hóa

0.13

0.87

0,00

so sánh

1.00

0,00

0,00

Xác suất logarit
chưa chuẩn hóa / logit

xác suất chưa
được chuẩn hóa

xác suất

Các vấn đề
chính xác

Bộ phân loại Softmax (Hồi quy logistic đa thức)



Muốn diễn giải điểm số phân loại thô dưới dạng xác suất

$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Chức năng

Tối đa hóa xác suất của lớp học chính xác

Tóm lại:

$$L_i = -\log P(Y = y_i | X = x_i)$$

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

con mèo

3.2

xe hơi

5.1

con ếch

-1,7

Bộ phân loại Softmax (Hồi quy logistic đa thức)



Muốn diễn giải điểm số phân loại thô dưới dạng xác suất

$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Chức năng

Tối đa hóa xác suất của lớp học chính xác

Tóm lại:

$$L_i = -\log P(Y = y_i | X = x_i)$$

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

con mèo

3.2

xe hơi

5.1

con ếch

-1,7

Câu 1: Giá trị tổn thất softmax tối thiểu/tối đa có thể là L_i là bao nhiêu ?

Q2: Khi khởi tạo, tất cả các s_j sẽ xấp xỉ bằng nhau; vậy tổn thất softmax L_i là bao nhiêu , giả sử có C lớp?

Bộ phân loại Softmax (Hồi quy logistic đa thức)



Muốn diễn giải điểm số phân loại thô dưới dạng xác suất

$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Chức năng

Tối đa hóa xác suất của lớp học chính xác

Tóm lại:

$$L_i = -\log P(Y = y_i | X = x_i)$$

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

con mèo

3.2

xe hơi

5.1

con ếch

-1,7

Câu 2: Khi khởi tạo, tất cả các giá trị

s sẽ xấp xỉ bằng nhau; vậy tổn thất là bao nhiêu?

A: $-\log(1/C) = \log(C)$,

Nếu $C = 10$, thì $L_i = \log(10) \approx 2,3$

Sắp tới:

- Điều chỉnh
- Tối ưu hóa

$$f(x, W) = Wx + b$$

